

Documentation technique

Module `src/Parsers`

Application de statistiques sportives

Projet 1A — 2026

Ce document décrit l'ensemble des classes et fonctions du dossier **`src/Parsers`**, responsable du chargement et du nettoyage des données sportives depuis les fichiers CSV.

Table des matières

1	Architecture générale	3
2	BaseLoader — Classe mère abstraite	3
2.1	Attributs de classe	3
2.2	Propriété root	3
2.3	Méthode _load_csv(filename, dtype, date_cols)	3
3	BadmintonLoader	4
3.1	load_players()	4
3.2	load_matches()	4
3.3	load_all()	4
4	BasketballLoader	4
4.1	load_players()	4
4.2	load_teams()	4
4.3	load_matches()	4
4.4	load_all()	4
4.5	get_team_roster(players, teams, team_name)	5
4.6	_height_to_cm(height_str)	5
5	ChessLoader	5
5.1	load_players()	5
5.2	load_matches()	5
5.3	load_all()	5
6	Cs2Loader	5
6.1	load_players()	5
6.2	load_coaches()	5
6.3	load_teams()	5
6.4	load_matches()	6
6.5	load_all()	6
7	FootballLoader	6
7.1	load_countries()	6
7.2	load_leagues()	6
7.3	load_teams()	6
7.4	load_players()	6
7.5	load_matches()	6
7.6	load_all()	6
8	FootballChampionsLeagueLoader	6
8.1	load_players()	7
8.2	load_teams()	7
8.3	load_matches()	7
8.4	load_all()	7
9	LolLoader	7
9.1	load_players()	7

9.2	load_coaches()	7
9.3	load_teams()	7
9.4	load_matches()	7
9.5	load_all()	7
10	Starcraft2Loader	8
10.1	load_players()	8
10.2	load_matches()	8
10.3	load_all()	8
11	TennisLoader	8
11.1	load_atp_players() / load_wta_players()	8
11.2	load_atp_matches() / load_wta_matches()	8
11.3	load_all()	8
11.4	_parse_dob(series)	8
11.5	_parse_tourney_date(series)	9
12	VolleyballLoader	9
12.1	load_countries()	9
12.2	load_players_men() / load_players_women()	9
12.3	load_coaches_men() / load_coaches_women()	9
12.4	load_matches_men()	9
12.5	load_matches_women()	9
12.6	load_all()	9
13	Synthèse des choix architecturaux	10

Architecture générale

Le dossier `src/Parsers` repose sur un pattern d'héritage unique : une classe mère `BaseLoader` centralise toute la mécanique commune, et chaque sport hérite d'elle en ajoutant uniquement ce qui lui est spécifique.

Classe	Sport couvert
<code>BaseLoader</code>	Classe abstraite — socle commun
<code>BadmintonLoader</code>	BWF World Tour
<code>BasketballLoader</code>	NBA 2022-2023
<code>ChessLoader</code>	FIDE (Échecs)
<code>Cs2Loader</code>	Counter-Strike 2
<code>FootballLoader</code>	Ligues européennes 2008-2016
<code>FootballChampionsLeagueLoader</code>	UEFA Champions League
<code>LolLoader</code>	League of Legends EMEA 2025
<code>Starcraft2Loader</code>	StarCraft II
<code>TennisLoader</code>	ATP et WTA 2024
<code>VolleyballLoader</code>	JO Paris 2024

Le fichier `__init__.py` expose les dix loaders comme imports directs du package, permettant d'écrire `from src.Parsers import BasketballLoader` plutôt que le chemin complet.

BaseLoader — Classe mère abstraite

Fichier : `src/Parsers/BaseLoader.py`

`BaseLoader` hérite de `ABC` (classe abstraite Python), ce qui interdit de l'instancier directement. Toutes les sous-classes doivent déclarer l'attribut `SPORT_FOLDER`.

Attributs de classe

- `_DATA_ROOT` — Chemin absolu vers le dossier `data/`, calculé dynamiquement depuis l'emplacement du fichier via `Path(__file__).parent.parent.parent / "data"`. Ce calcul garantit le fonctionnement quel que soit le répertoire de lancement.
- `SPORT_FOLDER` — Chaîne vide par défaut. Doit être redéfinie dans chaque sous-classe (ex : `"Basketball"`, `"tennis"`).

Propriété root

Retourne le chemin complet vers le dossier de données du sport : `_DATA_ROOT / SPORT_FOLDER`. Déclarée comme `@property`, elle s'utilise sans parenthèses (`self.root`).

Méthode `_load_csv(filename, dtype, date_cols)`

Méthode privée centrale du loader. Elle :

1. Lit le fichier CSV via `pd.read_csv()` en appliquant les types fournis dans `dtype`.
2. Convertit automatiquement chaque colonne listée dans `date_cols` via `pd.to_datetime(..., errors="coerce")` : une date malformée devient `NaT` sans lever d'exception.

BadmintonLoader

Fichier : `src/Parsers/BadmintonLoader.py`

Données : BWF World Tour — 88 joueurs, 226 matchs.

SPORT_FOLDER : `"badminton"`

`load_players()`

Charge `player.csv`. Colonnes retournées : `name`, `country`, `continent`. Aucune transformation appliquée.

`load_matches()`

Charge `match.csv` avec conversion de la colonne `date` en `datetime`. Colonnes : `tournament`, `city`, `country`, `date`, `tournament_type`, `round`, `player_1`, `player_2`, `winner`, `game_1_score`, `game_2_score`, `game_3_score`.

`load_all()`

Appelle `load_players()` et `load_matches()` et retourne le tuple `(players, matches)`.

BasketballLoader

Fichier : `src/Parsers/BasketballLoader.py`

Données : NBA saison 2022-2023 — 431 joueurs, 30 équipes.

SPORT_FOLDER : `"Basketball"`

`load_players()`

Charge `player.csv` et ajoute deux colonnes calculées :

- `full_name` — concaténation `first_name` + " " + `last_name`.
- `height_cm` — conversion du format américain pieds-pouces ("6-8") en centimètres via `_height_to_cm()`.

`load_teams()`

Charge `team.csv` avec typage forcé `id` → `int`. Colonnes : `id`, `full_name`, `abbreviation`, `nickname`, `city`, `state`.

`load_matches()`

Charge `game.csv` avec statistiques complètes par équipe : points, rebonds, passes décisives, tirs, lancers francs, etc.

`load_all()`

Retourne le tuple `(players, teams, matches)`.

`get_team_roster(players, teams, team_name)`

Méthode utilitaire. Recherche une équipe par nom complet, abréviation ou surnom (insensible à la casse), puis filtre les joueurs correspondants. Lève une `ValueError` si aucune équipe ne correspond.

`_height_to_cm(height_str)`

Méthode statique privée. Convertit le format pieds-pouces "6-8" en centimètres : $(pieds \times 12 + pouces) \times 2,54$. Retourne `None` si la valeur est manquante.

ChessLoader

Fichier : `src/Parsers/ChessLoader.py`

Données : FIDE — 206 joueurs, 256 matchs.

SPORT_FOLDER : "chess"

`load_players()`

Charge `player.csv` avec typage `Int64` (entier nullable) pour les colonnes `Elo` : `rating_standard`, `rating_rapid`, `rating_blitz`. Le type nullable gère proprement les joueurs sans classement dans une catégorie.

`load_matches()`

Charge `match.csv`. Particularité : la colonne `score_player_1` peut contenir la valeur textuelle "Bye" (joueur exempt de tour), ce qui justifie le typage `Float64` pour certaines colonnes.

`load_all()`

Retourne le tuple `(players, matches)`.

Cs2Loader

Fichier : `src/Parsers/Cs2Loader.py`

Données : Counter-Strike 2 — 160 joueurs, 32 coachs, 32 équipes, 106 matchs.

SPORT_FOLDER : "counter_strike_2"

`load_players()`

Charge `player.csv` avec conversion de `birthdate`. Colonnes : `pseudo`, `name`, `nationality`, `birthdate`, `role`, `team`.

`load_coaches()`

Charge `coach.csv` avec conversion de `birthdate`. Structure identique à `load_players()`.

`load_teams()`

Charge `team.csv`. Colonnes : `team`, `team_abbreviation`, `location`, `region`.

`load_matches()`

Charge `match.csv` avec conversion de date. Colonnes : `date`, `stage`, `round`, `best_of`, `team_1`, `team_2`, `score_team_1`, `score_team_2`.

`load_all()`

Retourne le tuple (`players`, `coaches`, `teams`, `matches`).

FootballLoader

Fichier : `src/Parsers/FootballLoader.py`

Données : Ligues européennes — 25 979 matchs sur les saisons 2008-2016.

SPORT_FOLDER : "Football"

C'est le loader avec le plus grand volume de données. Il charge cinq tables distinctes.

`load_countries()`

Charge `country.csv`. Colonnes : `id`, `name`.

`load_leagues()`

Charge `league.csv`. Colonnes : `id`, `country_id`, `name`.

`load_teams()`

Charge `team.csv`. Colonnes : `id`, `team_api_id`, `team_long_name`, `team_short_name`.

`load_players()`

Charge `player.csv`. Colonnes : `id`, `player_api_id`, `player_name`, `birthday`, `weight` (kg), `height` (cm).

`load_matches()`

Charge `match.csv` avec conversion de date. Contient les compositions d'équipe complètes (`home_player_1` à `home_player_11`).

`load_all()`

Retourne uniquement (`countries`, `leagues`, `matches`). Les tables joueurs et équipes sont disponibles via leurs méthodes dédiées mais non incluses dans `load_all()`.

FootballChampionsLeagueLoader

Fichier : `src/Parsers/FootballChampionsLeagueLoader.py`

Données : UEFA Champions League — 751 joueurs, 32 équipes, 125 matchs.

SPORT_FOLDER : "football_champions_league"

`load_players()`

Charge `player.csv` et corrige un problème dans le fichier source : deux colonnes portent le même nom `match_played`. Le loader les renomme en `match_played_field` et `match_played` via `rename()`. Des fautes de frappe d'origine (`balls_recoverd`, `cross_complted`) sont conservées volontairement pour rester fidèle au fichier source.

`load_teams()`

Charge `team.csv`. Colonnes : `full_name`, `short_name`, `year_founded`, `country`, `league`, `city`.

`load_matches()`

Charge `match.csv` avec conversion de date. Colonnes : `date`, `phase`, `round`, `matchday`, `group`, `team_home`, `team_away`, `scores`.

`load_all()`

Retourne le tuple (`players`, `teams`, `matches`).

LolLoader

Fichier : `src/Parsers/LolLoader.py`

Données : League of Legends EMEA 2025 — 50 joueurs, 25 coaches, 10 équipes, 45 matchs.

SPORT_FOLDER : "Lol"

`load_players()`

Charge `player.csv` avec conversion de `birthdate`. Colonnes : `pseudo`, `name`, `country_of_birth`, `birthdate`, `role`, `team`.

`load_coaches()`

Charge `coach.csv` avec conversion de `birthdate`. Structure identique à `load_players()`.

`load_teams()`

Charge `team.csv`. Colonnes : `team`, `team_abbreviation`, `location`, `region`.

`load_matches()`

Charge `match.csv` et ajoute une colonne calculée `duration_s` (durée de la partie en secondes), convertie depuis le format texte `HH:MM:SS` via `pd.to_timedelta().dt.total_seconds()`.

`load_all()`

Retourne le tuple (`players`, `coaches`, `teams`, `matches`).

Starcraft2Loader

Fichier : `src/Parsers/Starcraft2Loader.py`

Données : StarCraft II — 32 joueurs, 67 matchs.

SPORT_FOLDER : `"starcraft_2"`

`load_players()`

Charge `player.csv` avec conversion de `birthdate`. Colonnes : `pseudo`, `name`, `nationality`, `birthdate`, `race` (Terran / Zerg / Protoss), `team`.

`load_matches()`

Charge `match.csv` avec conversion de `date`. Colonnes : `date`, `round`, `group`, `best_of`, `player_1`, `player_2`, `score_player_1`, `score_player_2`.

`load_all()`

Retourne le tuple (`players`, `matches`).

TennisLoader

Fichier : `src/Parsers/TennisLoader.py`

Données : ATP et WTA 2024 — 443 joueurs ATP, 335 joueuses WTA, environ 5 700 matchs au total.

SPORT_FOLDER : `"tennis"`

Seul loader à gérer deux circuits en parallèle (ATP/WTA), ce qui double le nombre de méthodes.

`load_atp_players()` / `load_wta_players()`

Chargent les joueurs/joueuses, convertissent `dob` (format YYYYMMDD stocké comme float) via `_parse_dob()` et ajoutent `full_name`.

`load_atp_matches()` / `load_wta_matches()`

Chargent les matchs et convertissent `tourney_date` (format YYYYMMDD stocké comme int) via `_parse_tourney_date()`.

`load_all()`

Retourne le tuple (`atp_players`, `wta_players`, `atp_matches`, `wta_matches`).

`_parse_dob(series)`

Méthode statique privée. Convertit un float 19860504.0 en `datetime(1986, 5, 4)`. Utilise une chaîne `dropna().astype(int).astype(str)` pour éviter de planter sur les valeurs manquantes.

`_parse_tourney_date(series)`

Méthode statique privée. Même logique que `_parse_dob` mais depuis un entier directement.

VolleyballLoader

Fichier : `src/Parsers/VolleyballLoader.py`

Données : JO Paris 2024 — 156 joueurs H, 155 joueuses F, 39 coachs H, 42 coachs F, 26 matchs H, 26 matchs F.

SPORT_FOLDER : `"volleyball"`

Loader le plus volumineux avec sept tables et une distinction systématique hommes/femmes.

`load_countries()`

Charge la table des 224 codes IOC. Colonnes : `code`, `country`, `country_long`.

`load_players_men() / load_players_women()`

Chargent les joueurs/joueuses et ajoutent une colonne `gender` ("M" ou "F") pour permettre la fusion des deux tables sans ambiguïté.

`load_coaches_men() / load_coaches_women()`

Chargent les coachs masculins et féminins avec conversion de `birth_date`.

`load_matches_men()`

Charge `match_men.csv` et calcule automatiquement le gagnant en comparant `set_country_1` et `set_country_2`.

`load_matches_women()`

Même logique que pour les hommes, mais le fichier source utilise des noms de colonnes différents (`country_1` au lieu de `country_code_1`). Le loader les renomme pour assurer l'uniformité avec `load_matches_men()`.

`load_all()`

Retourne les sept tables dans l'ordre : (`countries`, `players_men`, `players_women`, `coaches_men`, `coaches_women`, `matches_men`, `matches_women`).

Synthèse des choix architecturaux

Choix	Bénéfice
Héritage de BaseLoader	Zéro duplication pour la lecture CSV et les dates
SPORT_FOLDER comme attribut de classe	Ajouter un sport = créer une sous-classe, pas modifier BaseLoader
_DATA_ROOT calculé dynamiquement	Fonctionne quel que soit le répertoire de lancement
errors="coerce" sur les dates	Données manquantes deviennent NaT sans exception
Méthodes statiques pour les parseurs de dates (Tennis)	Réutilisables sans instanciation, logique isolée et testable